



JavaScript Functions

Assoc. Prof. Dr. Kanda Runapongsa Saikaew
(krunapon@kku.ac.th)
Dept. of Computer Engineering
Khon Kaen University



Agenda

- Function expressions
- Arrow functions, the basics
- JavaScript specials



Function expressions

- The syntax that we used before is called a Function Declaration

```
1 function sayHi() {  
2   alert( "Hello" );  
3 }
```

- There is another syntax for creating a function that is called a Function Expression

```
1 let sayHi = function() {  
2   alert( "Hello" );  
3 };
```



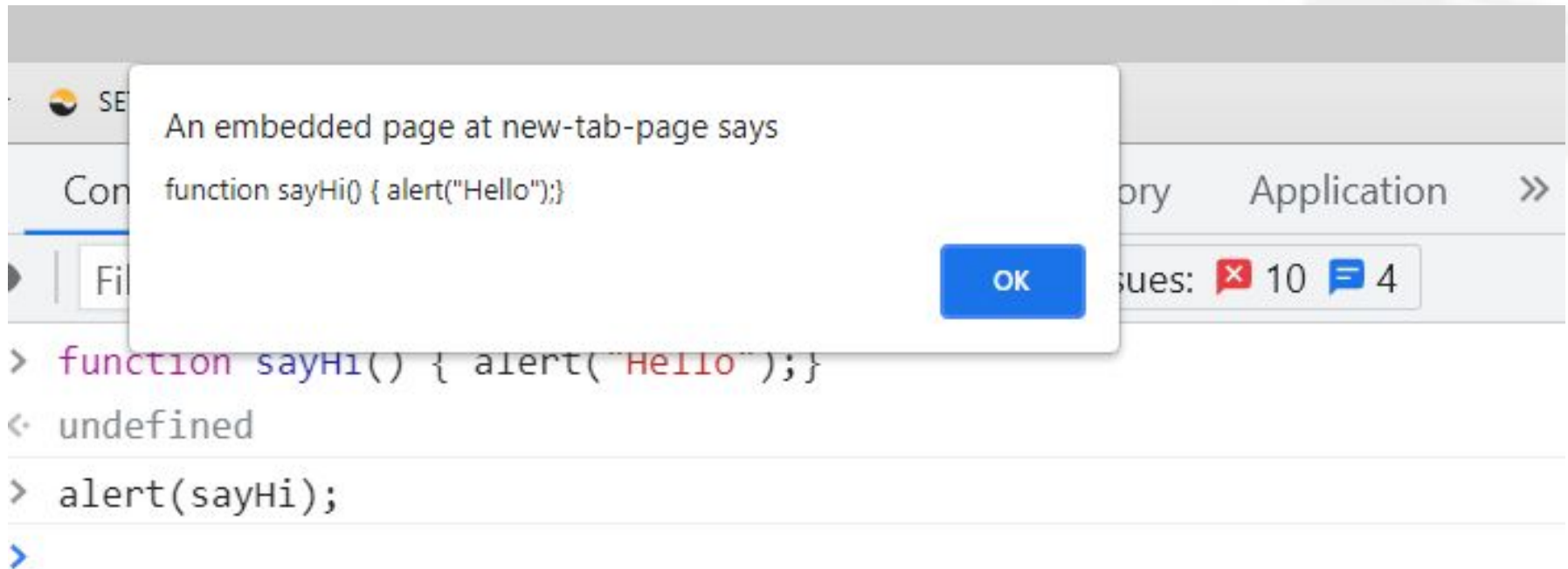
A function is a value

- No matter how the function is defined, it's just a value stored in the variable sayHi
- The meaning of these code samples is the same: “create a function and put it into the variables sayHi”

```
1 function sayHi() {  
2   alert( "Hello" );  
3 }  
4  
5 alert( sayHi ); // shows the function code
```



A function as a value



A function can be copied

- A function is a special value, in the sense that we can call it like sayHi()

```
1 function sayHi() { // (1) create
2   alert( "Hello" );
3 }
4
5 let func = sayHi; // (2) copy
6
7 func(); // Hello // (3) run the copy (it works)!
8 sayHi(); // Hello // this still works too (why wouldn't it)
```



Functions with and without parameters

JS func-as-value-params.js U X

javascript > demos > JS func-as-value-params.js > ...

```
1  const makeNoise = function() {  
2    console.log("Pling!");  
3  };  
4  
5  makeNoise(); // Pling!  
6  
7  const power = function(base, exponent) {  
8    let result = 1;  
9    for (let count = 0; count < exponent; count++) {  
10     result *= base;  
11   }  
12   return result;  
13 };  
14  
15 console.log(power(2,10)); // 1024
```



Declaration notation

- Function declarations are not part of the regular top-to-bottom flow of control
- They are conceptually moved to the top of their scope and can be used by all the code in that scope

JS declaration-notation.js U X

javascript > demos > JS declaration-notation.js >  future

```
1 console.log("The future says:", future());  
2 function future() {  
3     return "We'll need to keep learning and adapting"  
4 }
```



Callback function

- A callback function is a function passed into another function as an argument, which is then invoked inside the outer function to complete some kind of routine or action

```
callback.html x
callback.html > html > body > script > processUserInput > name
1  <!DOCTYPE html>
2  <html>
3    <body>
4      <script>
5        function greeting(name) {
6          alert('Hello ' + name);
7        }
8        function processUserInput(callback) {
9          let name = prompt("Please enter your name:");
10         callback(name);
11       }
12       processUserInput(greeting);
13     </script>
14   </body>
15 </html>
```

127.0.0.1:5500 says

Please enter your name:

OK Cancel

127.0.0.1:5500 says

Hello Manee

OK



Callbacks for processing numbers

JS filter-callback.js U X

javascript > demos > JS filter-callback.js > ...

```
1  let numbers = [1, 2, 4, 7, 3, 5, 6];
2
3  function isOddNumber(number) {
4      return number % 2;
5  }
6
7  const oddNumbers = numbers.filter(isOddNumber);
8  console.log(oddNumbers);
```

```
> let numbers = [1, 2, 4, 7, 3, 5, 6];

function isOddNumber(number) {
    return number % 2;
}

const oddNumbers = numbers.filter(isOddNumber);
console.log(oddNumbers);

▶ (4) [1, 7, 3, 5]
```

VM60:8



Using an anonymous function as a callback

JS anonymous-callback.js U X

javascript > demos > JS anonymous-callback.js > ...

```
1 let numbers = [1, 2, 4, 7, 3, 5, 6];
2
3 let oddNumbers = numbers.filter(function(number) {
4     return number % 2;
5 });
6 console.log()
```

```
> let numbers = [1, 2, 4, 7, 3, 5, 6];

function isOddNumber(number) {
    return number % 2;
}

const oddNumbers = numbers.filter(isOddNumber);
console.log(oddNumbers);

▶ (4) [1, 7, 3, 5]
```

VM60:8



Callback functions (1/2)

- We'll write a function ask(question, yes, no) with three parameters:
- question
 - Text of the question
- yes
 - Function to run if the answer is “Yes”
- no
 - Function to run if the answer is “No”



Callback functions (2/2)

- The arguments showOk and showCancel of ask are called *callback functions* or just *callbacks*.
- The idea is that we pass a function and expect it to be “called back” later if necessary
- In our case, showOk becomes the callback for “yes” answer, and showCancel for “no” answer.



Using Function Expressions

- Use Function Expressions to write the same function much shorter

```
> function yes() {  
    alert("You agreed");  
}  
  
function no() {  
    alert("You canceled the execution");  
}  
  
function ask(question, yes, no) {  
    if (confirm(question))  
        yes();  
    else  
        no();  
}  
  
ask("Do you agree?", yes, no);  
  
ask("Do you agree too?",  
    function() { alert("You agreed")},  
    function() { alert("You canceled the execution")});
```



Function expressions vs function declarations

- Function declarations and function expressions are 2 ways to create function using the function keyword

```
> function sumA(a, b) {  
  return a + b;  
}
```

```
(function sumB(a, b) {  
  return a + b;  
});
```

```
< f sumB(a, b) {  
  return a + b;  
}
```

```
> sumA(1,2);
```

```
< 3
```

```
> sumB(1,2);
```

```
✖ ▶ Uncaught ReferenceError: sumB is not defined  
   at <anonymous>:1:1
```

```
> |
```



Function Expression vs Function Declaration (1/2)

- Function Declaration: a function, declared as a separate statement, in the main code flow

```
1 // Function Declaration
2 function sum(a, b) {
3     return a + b;
4 }
```

- Function Expression: a function, created inside an expression or inside another syntax construct
- Here, the function is created at the right side of the “assignment expression” =

```
1 // Function Expression
2 let sum = function(a, b) {
3     return a + b;
4 };
```



Function Expression vs Function Declaration (2/2)

- A function expression is created and can be used (assigned, called, etc.) from now on
- A function declaration can be called earlier than it is defined

```
1 sayHi("John"); // Hello, John
2
3 function sayHi(name) {
4   alert( `Hello, ${name}` );
5 }
```

```
1 sayHi("John"); // error!
2
3 let sayHi = function(name) {
4   alert( `Hello, ${name}` );
5 };
```



Using Function declaration

- Using function declaration for conditional cases

```
1  let age = prompt("What is your age?", 18);
2
3  // conditionally declare a function
4  if (age < 18) {
5
6      function welcome() {
7          alert("Hello!");
8      }
9
10 } else {
11
12     function welcome() {
13         alert("Greetings!");
14     }
15
16 }
17
18 // ...use it later
19 welcome(); // Error: welcome is not defined
```



Using let and function expression

- Using a Function Expression and assign welcome to the variable that is declared outside of it and has the proper visibility

```
1  let age = prompt("What is your age?", 18);
2
3  let welcome;
4
5  if (age < 18) {
6
7      welcome = function() {
8          alert("Hello!");
9      };
10
11 } else {
12
13     welcome = function() {
14         alert("Greetings!");
15     };
16
17 }
18
19 welcome(); // ok now
```



Using a question mark operator ?:

```
1 let age = prompt("What is your age?", 18);  
2  
3 let welcome = (age < 18) ?  
4   function() { alert("Hello!"); } :  
5   function() { alert("Greetings!"); };  
6  
7 welcome(); // ok now
```



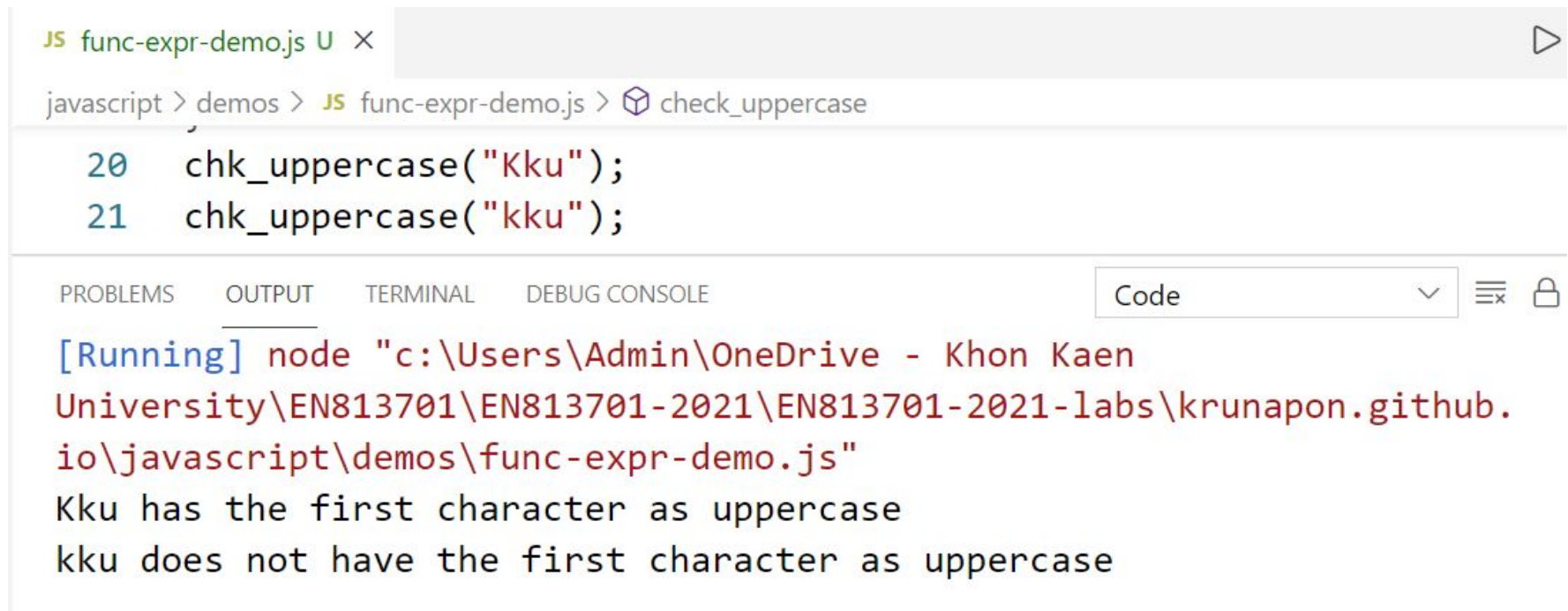
When to Choose Function Declaration vs Function Expression?

- When we need to declare a function, the first to consider is Function Declaration syntax
 - It gives more freedom in how to organize our code, because we can call such functions before they are declared
- That's also better for readability, as it's easier to look up function `f(...) {...}` in the code than `let f = function(...) {...};`
 - Function Declarations are more “eye-catching”
- For a conditional declaration, function Expression should be used



Demo Problem 1

- Define function expression to check whether the first character is uppercase or not



```
JS func-expr-demo.js U X
javascript > demos > JS func-expr-demo.js > check_uppercase

20  chk_uppercase("Kku");
21  chk_uppercase("kku");
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE Code

[Running] node "c:\Users\Admin\OneDrive - Khon Kaen University\EN813701\EN813701-2021\EN813701-2021-labs\krunapon.github.io\javascript\demos\func-expr-demo.js"

Kku has the first character as uppercase

kku does not have the first character as uppercase



Demo Problem 1 Solution

demos > javascripts > JS regexp_functions.js > [🔗] chk_uppercase

```
2   const regex = /^[A-Z]/;
3   if (regex.test(str)) {
4       console.log(`${str}'s first character is an uppercase`);
5   } else {
6       console.log(`${str}'s first character is not an uppercase`);
7   }
8 }
9
10  check_uppercase("Abcd");
11  check_uppercase("abcd");
12
13  let chk_uppercase = function check_uppercase(str) {
14      const regex = /^[A-Z]/;
15      if (regex.test(str)) {
16          console.log(`${str}'s first character is an uppercase`);
17      } else {
18          console.log(`${str}'s first character is not an uppercase`);
19      }
```



Summary

- Functions are values
- They can be assigned, copied, or declared in any place of the code
- If the function is declared as a separate statement in the main code flow, that's called a "Function Declaration"
- If the function is created as a part of an expression, it's called a "Function Expression"
- Function Declarations are processed before the code block is executed. They are visible everywhere in the block
- Function Expressions are created when the execution flow reaches them



Agenda

- Function expressions
- **Arrow functions, the basics**
- JavaScript specials



Arrow functions, the basics

- There's another very simple and concise syntax for creating functions
- It's called “arrow functions”, because it looks like this:

```
1 let func = (arg1, arg2, ...argN) => expression
```

- In other words, it's the shorter version of

```
1 let func = function(arg1, arg2, ...argN) {  
2   return expression;  
3 };
```



A concrete example of arrow function

- As you can see $(a, b) \Rightarrow a + b$ means a function that accepts two arguments named a and b
- Upon the execution, it evaluates the expression $a + b$ and returns the result

```
JS arrow-func-sum.js U X
javascript > demos > JS arrow-func-sum.js > sum1
1 // function declaration
2 function sum1(a, b) {
3     return a + b;
4 }
5 m = 2;
6 n = 3;
7 console.log("Sum of " + m + " and " + n + " is " + sum1(m, n));
8
9 // function expression
10 let sum2 = function(a, b) {
11     return a + b;
12 }
13 console.log("Sum of " + m + " and " + n + " is " + sum2(m, n));
14
15 // arrow function
16 let sum3 = (a, b) => a + b;
17 console.log("Sum of " + m + " and " + n + " is " + sum3(m, n));
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

[Running] node "c:\Users\Admin\OneDrive - Khon Kaen University\EN813701\EN813701-2021\EN813701-2021-labs\krunapon.github.io\jav
Sum of 2 and 3 is 5
Sum of 2 and 3 is 5
Sum of 2 and 3 is 5

Arrow function with no argument or only one argument

- If there are no arguments, parentheses will be empty (but they should be present)

```
1 let sayHi = () => alert("Hello!");  
2  
3 sayHi();
```

- If we have only one argument, then parentheses around parameters can be omitted, making that even shorter

```
1 let double = n => n * 2;  
2 // roughly the same as: let double = function(n) { return n * 2 }  
3  
4 alert( double(3) ); // 6
```



Multiline arrow functions

- Sometimes we need something a little bit more complex, like multiple expressions or statements
- It is also possible, but we should enclose them in curly braces
- Then use a normal return within them

```
1 let sum = (a, b) => { // the curly brace opens a multiline function
2   let result = a + b;
3   return result; // if we use curly braces, then we need an explicit "return"
4 };
5
6 alert( sum(1, 2) ); // 3
```



Rewriting with arrow functions



The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following code and its output:

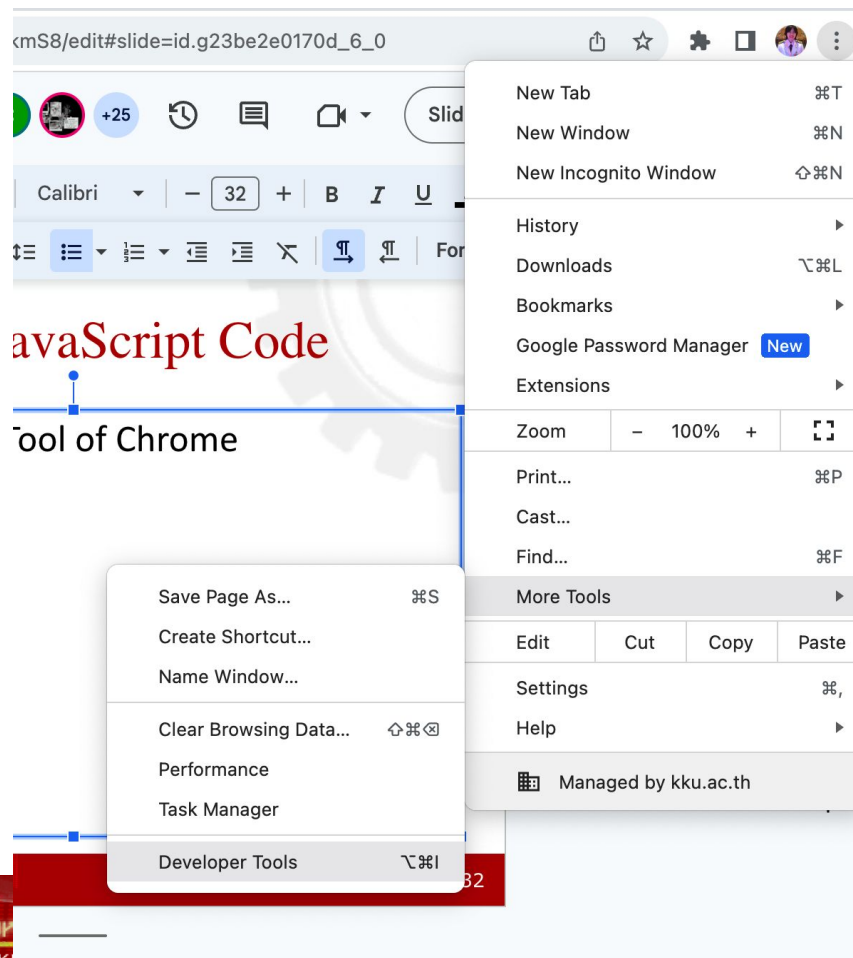
```
> function ask(question, yes, no) {  
    if (confirm(question))  
        yes();  
    else  
        no();  
}  
  
ask("Do you agree?", function() {alert("You agreed")}, function() {alert("You canceled")});  
  
ask("Do you agree too?", () => alert("You agreed"), () => alert("you canceled"))  
< undefined  
>
```

The console interface includes tabs for Console, Sources, Network, Performance, Memory, and Application. The Console tab shows a filter input, a 'Default levels' dropdown, and a message count of '2 Issues: 2'.



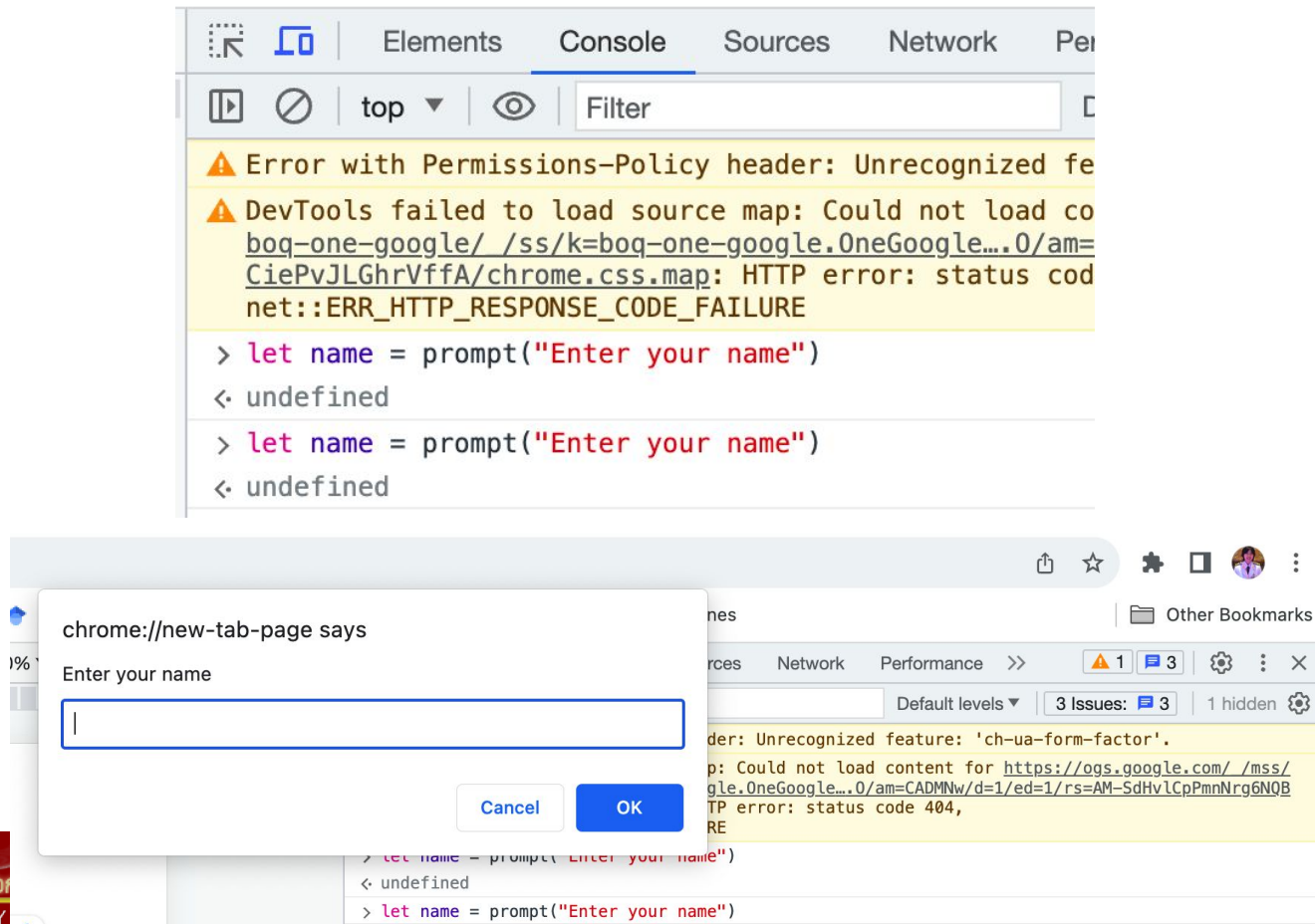
Using Chrome to Test JavaScript (1/2)

- Go to Three dots > More Tools > Developer Tools



Using Chrome to Test JavaScript (2/2)

- Open tab Console and then insert the code in the Console part



Referring JavaScript Code from an HTML file

<> demo.html U X



javascript > demos > <> demo.html > head > script

```
1  <!DOCTYPE html>
2  <head>
3      <script>
4          let num1 = prompt("Enter the first number");
5          let num2 = prompt("Enter the second number");
6          sum = Number(num1) + Number(num2);
7          alert(`${num1} + ${num2} = ${sum}`)
8      </script>
9  </head>
10 </html>
```



Demo Problem 2

Rewriting this code using conditional operator and arrow function

```
let age = prompt("What is your age?", 18);
let welcome;
if (age < 18) {
    welcome = function() {
        alert("Hello!");
    };
} else {
    welcome = function() {
        alert("Greetings!");
    };
}
welcome(); // ok now
```



Demo Problem 2 Solution

```
javascript > demos > JS arrow-sol-exercise.js > ...
```

```
You, 14 months ago | 2 authors (You and one other)
```

```
1 let age = prompt("What is your age?", 18);
2 let welcome = age < 18 ? () => alert("Hello!") : () => alert("Greetings!");
3 welcome();
4
```

- We've replaced the **if...else** statement with a conditional (ternary) operator **?:**. This operator provides a concise way to write simple if-else statements.
- We've used arrow functions **() => ...** instead of the traditional function declarations. Arrow functions provide a shorter syntax and lexically bind **this**.
- The entire welcome function assignment is now a single line, with the condition **(age < 18)** determining which arrow function is assigned.



Demo Problem 3

- Define an arrow function `divideByTwo` which accepts an integer and returns that number divided by 2.
- The program also checks whether the entered text is an integer or not
- The program will keep asking the user until a user enters a valid integer



Demo Problem 3 Interaction



Demo Problem 3 Solution

```
javascript > demos > JS arrow-divide-by-two.js > ...
```

You, 12 months ago | 2 authors (Kanda and one other)

```
1 let divideByTwo = n => (n / 2)      Kanda, 3 years
2 do {
3     inputInt = Number(prompt("Enter an integer"))
4     if (inputInt && Number.isInteger(inputInt)) {
5         alert(divideByTwo(inputInt));
6         break;
7     }
8 } while (true);
```



Summary

- Arrow functions may appear unfamiliar and not very readable at first, but that quickly changes as the eyes get used to the structure.
- They are very convenient for simple one-line actions, when we're just too lazy to write many words.
- Arrow functions come in two flavors
 - Without curly braces: `(...args) => expression`
 - The right side is an expression: the function evaluates it and returns the result
 - With curly braces: `(...args) => { body }`
 - Brackets allow us to write multiple statements but we need return to return something



References

- <https://javascript.info/>
- <https://cs.lmu.edu/~ray/notes/javascriptfunctions/>
- https://eloquentjavascript.net/03_functions.html
- https://developer.mozilla.org/en-US/docs/Glossary/Callback_function
- <https://www.javascripttutorial.net/javascript-callback/>
- <https://dmitripavlutin.com/javascript-function-expressions-and-declarations/>

